Fix C++26 by making the rank-1, rank-2, rank-k, and rank-2k updates consistent with the BLAS

Document #:	P3371R1
Date:	2024-09-11
Project:	Programming Language C++ LEWG

Reply-to:	Mark Hoemmen <mhoemmen@nvidia.com> Ilya Burylov <burylov@gmail.com>

Contents

§ 1 Authors

— Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

— Ilya Burylov (burylov@gmail.com) (NVIDIA)

§ 2 Revision history

— Revision 0 to be submitted 2024-08-15

— Revision 1 to be submitted 2024-09-15

  — Main wording changes from R0:

    — Instead of just changing the symmetric and Hermitian rank-k and rank-2k updates to have both overwriting and updating overloads, change *all* the update functions in this way: rank-1 (general, symmetric, and Hermitian), rank-2, rank-k, and rank-2k

    — For all the new updating overloads, specify that `C` (or `A`) may alias `E`

    — For the new symmetric and Hermitian updating overloads, specify that the functions access the new `E` parameter in the same way (e.g., with respect to the lower or upper triangle) as the `C` (or `A`) parameter

    — Add exposition-only concept *`noncomplex`* to constrain a scaling factor to be noncomplex, as needed for Hermitian rank-1 and rank-k functions

  — Add Ilya Burylov as coauthor

  — Change title and abstract to express the wording changes

  — Add nonwording section explaining why we change rank-1 and rank-2 updates to be consistent with rank-k and rank-2k updates

  — Add nonwording sections explaining why we don't change `hermitian_matrix_vector_product`, `hermitian_matrix_product`, `triangular_matrix_product`, or the triangular solves

  — Add nonwording section explaining why we constrain some scaling factors to be noncomplex at compile time, instead of taking a run-time approach

  — Reorganize and expand nonwording sections

§ 3 Abstract

The [linalg] functions `matrix_rank_1_update`, `matrix_rank_1_update_c`, `symmetric_rank_1_update`, `hermitian_rank_1_update`,

`symmetric_matrix_rank_k_update`, `hermitian_matrix_rank_k_update`, `symmetric_matrix_rank_2k_update`, and `hermitian_matrix_rank_2k_update` currently have behavior inconsistent with their corresponding BLAS (Basic Linear Algebra Subroutines) routines. Also, the behavior of the rank-k and rank-2k updates is inconsistent with that of `matrix_product`, even though in mathematical terms they are special cases of a matrix-matrix product. We propose three fixes.

1. Add "updating" overloads to the rank-1, rank-2, rank-k, and rank-2k update functions. The new overloads are analogous to the updating overloads of `matrix_product`. For example, `symmetric_matrix_rank_k_update(A, scaled(beta, C), C, upper_triangle)` will perform $C := \beta C + AA^T$.

2. Change the behavior of the existing rank-1, rank-2, rank-k, and rank-2k update functions to be "overwriting" instead of "unconditionally updating." For example, `symmetric_matrix_rank_k_update(A, C, upper_triangle)` will perform $C = AA^T$ instead of $C := C + AA^T$.

3. For `hermitian_rank_1_update` and `hermitian_rank_k_update`, constrain the `Scalar` template parameter (if any) to be noncomplex. This ensures that the update will be mathematically Hermitian. (A constraint is not needed for the rank-2 and rank-2k update functions.)

Items (2) and (3) are breaking changes to the current Working Draft. Thus, we must finish this before finalization of C++26.

§ 4 Discussion and proposed changes

§ 4.1 Support both overwriting and updating rank-k and rank-2k updates

§ 4.1.1 BLAS supports scaling factor beta; std::linalg currently does not

Each function in any section whose label begins with "linalg.algs" generally corresponds to one or more routines or functions in the original BLAS (Basic Linear Algebra Subroutines). Every computation that the BLAS can do, a function in the C++ Standard Library should be able to do.

One `std::linalg` user reported an exception to this rule. The BLAS routine `DSYRK` (Double-precision SYmmetric Rank-K update) computes $C := \beta C + \alpha AA^T$, but the corresponding `std::linalg` function `symmetric_matrix_rank_k_update` only computes $C := C + \alpha AA^T$. That is, `std::linalg` currently has no way to express this BLAS operation with a general β scaling factor. This issue applies to all of the symmetric and Hermitian rank-k and rank-2k update functions.

— `symmetric_matrix_rank_k_update`: computes $C := C + \alpha AA^T$

— `hermitian_matrix_rank_k_update`: computes $C := C + \alpha A A^H$

— `symmetric_matrix_rank_2k_update`: computes $C := C + \alpha A B^H + \alpha B A^H$

— `hermitian_matrix_rank_2k_update`: computes $C := C + \alpha A B^H + \bar{\alpha} B A^H$, where $\bar{\alpha}$ denotes the complex conjugate of α

§ 4.1.2 Make these functions consistent with general matrix product

These functions implement special cases of matrix-matrix products. The `matrix_product` function in `std::linalg` implements the general case of matrix-matrix products. This function corresponds to the BLAS's `SGEMM`, `DGEMM`, `CGEMM`, and `ZGEMM`, which compute $C := \beta C + \alpha A B$, where α and β are scaling factors. The `matrix_product` function has two kinds of overloads:

1. *overwriting* ($C = AB$) and

2. *updating* ($C = E + AB$).

The updating overloads handle the general α and β case by `matrix_product(scaled(alpha, A), B, scaled(beta, C), C)`. The specification explicitly permits the input `scaled(beta, C)` to alias the output `C` (**[linalg.algs.blas3.gemm]** 10: "*Remarks*: `C` may alias `E`"). The `std::linalg` library provides overwriting and updating overloads so that it can do everything that the BLAS does, just in a more idiomatically C++ way. Please see P1673R13 Section 10.3 ("Function argument aliasing and zero scalar multipliers") for a more detailed explanation.

§ 4.1.3 Fix requires changing behavior of existing overloads

The problem with the current symmetric and Hermitian rank-k and rank-2k functions is that they have the same *calling syntax* as the overwriting version of `matrix_product`, but *semantics* that differ from both the overwriting and the updating versions of `matrix_product`. For example,

```
hermitian_matrix_rank_k_update(alpha, A, C);
```

updates C with $C + \alpha A A^H$, but

```
matrix_product(scaled(alpha, A), conjugate_transposed(A), C);
```

overwrites C with $\alpha A A^H$. The current rank-k and rank-2k overloads are not overwriting, so we can't just fix this problem by introducing an "updating" overload for each function.

Incidentally, the fact that these functions have "update" in their name is not relevant, because that naming choice is original to the BLAS. The BLAS calls its corresponding `xSYRK`, `xHERK`, `xSYR2K`, and `xHER2K` routines "{Symmetric, Hermitian} rank {one, two} update," even though setting $\beta = 0$ makes these routines "overwriting" in the sense of `std::linalg`.

§ 4.1.4 Add new updating overloads; make existing ones overwriting

We propose to fix this by making the functions work just like `matrix_vector_product` or `matrix_product`. This entails three changes.

1. Add two new exposition-only concepts *`possibly-packed-in-matrix`* and *`possibly-packed-out-matrix`* for constraining input and output parameters of the changed or new symmetric and Hermitian update functions.

2. Add "updating" overloads of the symmetric and Hermitian rank-k and rank-2k update functions.

    a. The updating overloads take a new input matrix parameter `E`, analogous to the updating overloads of `matrix_product`, and make `C` an output parameter instead of an in/out parameter. For example, `symmetric_matrix_rank_k_update(A, E, C, upper_triangle)` computes $C = E + AA^T$.

    b. Explicitly permit `C` and `E` to alias, thus permitting the desired case where `E` is `scaled(beta, C)`.

    c. The updating overloads take `E` as a *`possibly-packed-in-matrix`*, and take `C` as a *`possibly-packed-out-matrix`* (instead of a *`possibly-packed-inout-matrix`*).

    d. `E` must be accessed as a symmetric or Hermitian matrix (depending on the function name) and such accesses must use the same triangle as `C`. (The existing [linalg.general] 4 wording for symmetric and Hermitian behavior does not cover `E`.)

3. Change the behavior of the existing symmetric and Hermitian rank-k and rank-2k overloads to be overwriting instead of updating.

    a. For example, `symmetric_matrix_rank_k_update(A, C, upper_triangle)` will compute $C = AA^T$ instead of $C := C + AA^T$.

    b. Change `C` from a *`possibly-packed-inout-matrix`* to a *`possibly-packed-out-matrix`*.

Items (2) and (3) are breaking changes to the current Working Draft. This needs to be so that we can provide the overwriting behavior $C := \alpha AA^T$ or $C := \alpha AA^H$ that the corresponding BLAS routines already provide. Thus, we must finish this before finalization of C++26.

Both sets of overloads still only write to the specified triangle (lower or upper) of the output matrix `C`. As a result, the new updating overloads only read from that triangle of the input matrix `E`. Therefore, even though `E` may be a different matrix than `C`, the updating overloads do not need an additional `Triangle t_E` parameter for `E`. The `symmetric_*`

functions interpret `E` as symmetric in the same way that they interpret `C` as symmetric, and the `hermitian_*` functions interpret `E` as Hermitian in the same way that they interpret `C` as Hermitian. Nevertheless, we do need new wording to explain how the functions may interpret and access `E`.

§ 4.2 Change rank-1 and rank-2 updates to be consistent with rank-k and rank-2k

1. Rank-1 and rank-2 updates currently unconditionally update and do not take a β scaling factor.

2. We propose making all the rank-1 and rank-2 update functions consistent with the proposed change to the rank-k and rank-2k updates. This means both changing the meaning of the current overloads to be overwriting, and adding new overloads that are updating. This includes general (nonsymmetric), symmetric, and Hermitian rank-1 update functions, as well as symmetric and Hermitian rank-2 update functions.

3. As a result, the exposition-only concept *`possibly-packed-inout-matrix`* is no longer needed. We propose removing it.

§ 4.2.1 Current std::linalg behavior

The rank-k and rank-2k update functions have the following rank-1 and rank-2 analogs, where A denotes a symmetric or Hermitian matrix (depending on the function's name) and x and y denote vectors.

— `symmetric_matrix_rank_1_update`: computes $A := A + \alpha x x^T$

— `hermetian_matrix_rank_1_update`: computes $A := A + \alpha x x^H$

— `symmetric_matrix_rank_2_update`: computes $A := A + \alpha x y^T + \alpha y x^T$

— `hermitian_matrix_rank_2_update`: computes $A := A + \alpha x y^H + \bar{\alpha} x y^H$

These functions *unconditionally* update the matrix A. They do not have an overwriting option. In this, they follow the "general" (not necessarily symmetric or Hermitian) rank-1 update functions.

— `matrix_rank_1_update`: computes $A := A + x y^T$

— `matrix_rank_1_update_c`: computes $A := A + x y^H$

§ 4.2.2 Current behavior is inconsistent with BLAS Standard and rank-k and rank-2k updates

These six rank-1 and rank-2 update functions map to BLAS routines as follows.

— `matrix_rank_1_update`: `xGER`

— `matrix_rank_1_update`: `xGERC`

— `symmetric_matrix_rank_1_update`: `xSYR`, `xSPR`

— `hermitian_matrix_rank_1_update`: `xHER`, `xHPR`

— `hermitian_matrix_rank_1_update`: `xSYR2`, `xSPR2`

— `hermitian_matrix_rank_1_update`: `xHER2`, `xHPR2`

The Reference BLAS and the BLAS Standard (see Chapter 2, pp. 64 - 68) differ here. The Reference BLAS and the original 1988 BLAS 2 paper specify all of the rank-1 and rank-2 update routines listed above as unconditionally updating, and not taking a β scaling factor. However, the (2002) BLAS Standard specifies all of these rank-1 and rank-2 update functions as taking a β scaling factor. We consider the latter to express our design intent. It is also consistent with the corresponding rank-k and rank-2k update functions in the BLAS, which all take a β scaling factor and thus can do either overwriting (with zero β) or updating (with nonzero β). These routines include `xSYRK`, `xHERK`, `xSYR2K`, and `xHER2K`. One could also include the general matrix-matrix product `xGEMM` among these, as `xGEMM` also takes a β scaling factor.

§ 4.2.3 This change would remove a special case in std::linalg's design

Section 10.3 of P1673R13 explains the three ways that the std::linalg design translates Fortran `INTENT(INOUT)` arguments into a C++ idiom.

1. Provide in-place and not-in-place overloads for triangular solve and triangular multiply.

2. "Else, if the BLAS function unconditionally updates (like `xGER`), we retain read-and-write behavior for that argument."

3. "Else, if the BLAS function uses a scalar beta argument to decide whether to read the output argument as well as write to it (like `xGEMM`), we provide two versions: a write-only version (as if `beta` is zero), and a read-and-write version (as if `beta` is nonzero)."

Our design goal was for functions with vector or matrix output to imitate `std::transform` as much as possible. This favors Way (3) as the default approach, which turns `INTENT(INOUT)` arguments on the Fortran side into separate input and output parameters on the C++ side. Way (2) is really an awkward special case. The BLAS Standard effectively eliminates this special case on the Fortran side by making the rank-1 and rank-2 updates work just like the rank-k and rank-2k updates, with a β scaling factor. This makes it natural to eliminate the Way (2) special case on the C++ side as well.

§ 4.2.4 Exposition-only concept no longer needed

These changes make the exposition-only concept *`possibly_packed_inout_matrix`* superfluous. We propose removing it.

Note that this would not eliminate all uses of the exposition-only concept *inout-matrix*. The in-place triangular matrix product functions `triangular_matrix_left_product` and `triangular_matrix_right_product`, and the in-place triangular linear system solve functions `triangular_matrix_matrix_left_solve` and `triangular_matrix_matrix_right_solve` would still use *inout-matrix*.

§ 4.3 Constrain alpha in Hermitian rank-1 and rank-k updates to be noncomplex

§ 4.3.1 Scaling factor alpha needs to be noncomplex, else update may be non-Hermitian

The C++ Working Draft already has `Scalar alpha` overloads of `hermitian_rank_k_update`. The `Scalar` type currently can be complex. However, if `alpha` has nonzero imaginary part, then αAA^H may no longer be a Hermitian matrix, even though AA^H is mathematically always Hermitian. For example, if A is the identity matrix (with ones on the diagonal and zeros elsewhere) and $\alpha = i$, then αAA^H is the diagonal matrix whose diagonal elements are all i. While that matrix is symmetric, it is not Hermitian, because all elements on the diagonal of a Hermitian matrix must have nonzero imaginary part. The rank-1 update function `hermitian_rank_1_update` has the analogous issue.

The BLAS solves this problem by having the Hermitian rank-1 update routines `xHER` and rank-k update routines `xHERK` take the scaling factor α as a noncomplex number. This suggests a fix: For all `hermitian_rank_1_update` and `hermitian_rank_k_update` overloads that take `Scalar alpha`, constrain `Scalar` so that it is noncomplex. We can avoid introducing new undefined behavior (or "valid but unspecified" elements of the output matrix) by making "noncomplex" a constraint on the `Scalar` type of `alpha`. "Noncomplex" should follow the definition of "noncomplex" used by *conj-if-needed*: either an arithmetic type, or `conj(E)` is not ADL-findable for an expression `E` of type `Scalar`.

§ 4.3.2 Nothing wrong with rank-2 or rank-2k updates

This issue does *not* arise with the rank-2 or rank-2k updates. In the BLAS, the rank-2 updates `xHER2` and the rank-2k updates `xHER2K` all take `alpha` as a complex number. The matrix $\alpha AB^H + \bar{\alpha} BA^H$ is Hermitian by construction, so there's no need to impose a precondition on the value of α.

§ 4.3.3 Nothing wrong with scaling factor beta

Both the current wording and the proposed changes to all these update functions behave correctly with respect to `beta`.

For the new updating overloads of `hermitian_rank_1_update` and `hermitian_rank_k_update`, [linalg] expresses a `beta` scaling factor by letting users

supply `scaled(beta, C)` as the argument for `E`. The wording only requires that `E` be Hermitian. If `E` is `scaled(beta, C)`, this concerns only the product of `beta` and `C`. It would be incorrect to constrain `beta` or `C` separately. For example, if $\beta = -i$ and C is the matrix whose elements are all i, then C is not Hermitian but βC (and therefore `scaled(beta, C)`) is Hermitian.

This issue does *not* arise with the rank-2k updates. For example, the BLAS routine `xHER2K` takes `beta` as a real number. The previous paragraph's reasoning for `beta` applies here as well.

This issue also does not arise with the rank-2 updates. In the Reference BLAS, the rank-2 update routines `xHER2` do not have a way to supply `beta`, while in the BLAS Standard, `xHER2` *does* take `beta`. The BLAS Standard says that "α is a complex scalar and and [sic] β is a real scalar." The Fortran 77 and C bindings specify the type of `beta` as real (`<rtype>` resp. `RSCALAR_IN`), but the Fortran 95 binding lists both `alpha` and `beta` as `COMPLEX(<wp>)`. The type of `beta` in the Fortran 95 is likely a typo, considering the wording.

§ 4.3.4 Nothing wrong with Hermitian matrix-vector and matrix-matrix products

In our view, the current behavior of `hermitian_matrix_vector_product` and `hermitian_matrix_product` is correct with respect to the `alpha` scaling factor. These functions do not need extra overloads that take `Scalar alpha`. Users who want to supply `alpha` with nonzero imaginary part should *not* scale the matrix `A` (as in `scaled(alpha, A)`). Instead, they should scale the input vector `x`, as in the following.

```
        auto alpha = std::complex{0.0, 1.0};
        hermitian_matrix_vector_product(A, upper_triangle, scaled(alpha,
x), y);
```

§ 4.3.4.1 In BLAS, matrix is Hermitian, but scaled matrix need not be

In Chapter 2 of the BLAS Standard, both `xHEMV` and `xHEMM` take the scaling factors α and β as complex numbers (`COMPLEX<wp>`, where `<wp>` represents the current working precision). The BLAS permits `xHEMV` or `xHEMM` to be called with `alpha` whose imaginary part is nonzero. The matrix that the BLAS assumes to be Hermitian is A, not αA. Even if A is Hermitian, αA might not necessarily be Hermitian. For example, if A is the identity matrix (diagonal all ones) and α is i, then αA is not Hermitian but skew-Hermitian.

The current [linalg] wording requires that the input matrix be Hermitian. This excludes using `scaled(alpha, A)` as the input matrix, where `alpha` has nonzero imaginary part. For example, the following gives mathematically incorrect results.

```
        auto alpha = std::complex{0.0, 1.0};
        hermitian_matrix_vector_product(scaled(alpha, A), upper_triangle,
x, y);
```

Note that the behavior of this is still well defined, at least after applying the fix proposed in LWG4136 for diagonal elements with nonzero imaginary part. It does not violate a precondition. Therefore, the Standard has no way to tell the user that they did something wrong.

§ 4.3.4.2 Status quo permits scaling via the input vector

The current wording permits introducing the scaling factor `alpha` through the input vector.

```
        auto alpha = std::complex{0.0, 1.0};
        hermitian_matrix_vector_product(A, upper_triangle, scaled(alpha,
x), y);
```

This is fine as long as αAx equals $A\alpha x$, that is, as long as α commutes with the elements of A. This issue would only be of concern if those multiplications might be noncommutative. We're already well outside the world of "types that do ordinary arithmetic with `std::complex`." This scenario would only arise given a user-defined complex type, like `user_complex<user_noncommutative>` in the example below, whose real parts have noncommutative multiplication.

```
        template<class T>
        class user_complex {
        public:
          user_complex(T re, T im) : re_(re), im_(im) {}

          // ... overloaded arithmetic operators ...

          friend T real(user_complex<T> z) { return z.re_; }
          friend T imag(user_complex<T> z) { return z.im_; }
          friend user_complex<T> conj(user_complex<T> z) {
            return {real(z), -imag(z)};
          }

        private:
          T re_, im_;
        };

        auto alpha = user_complex<user_noncommutative>{something,
something_else};
        hermitian_matrix_vector_product(N, upper_triangle, scaled(alpha,
x), y);
```

The [linalg] library was designed to support element types with noncommutative multiplication. On the other hand, generally, if we speak of Hermitian matrices or even of inner products (which are used to define Hermitian matrices), we're working in a vector space. This means that multiplication of the matrix's elements is commutative. Anything more general than that is far beyond what the BLAS can do. Thus, we think restricting use of `alpha` with nonzero imaginary part to `scaled(alpha, x)` is not so onerous.

§ 4.3.4.3 Scaling via the input vector is weird, but the least bad choice

Many users may not like the status quo of needing to scale `x` instead of `A`. First, it differs from the BLAS, which puts `alpha` before `A` in its `xHEMV` and `xHEMM` function arguments. Second, users would get no compile-time feedback and likely no run-time feedback if they scale `A` instead of `x`. Their code would compile and produce correct results for almost all matrix-vector or matrix-matrix products, *except* for the Hermitian case, and *only* when the scaling factor has a nonzero imaginary part. However, we still think the status quo is the least bad choice. We explain why by discussing the following alternatives.

1. Treat `scaled(alpha, A)` as a special case: expect `A` to be Hermitian and permit `alpha` to have nonzero imaginary part

2. Forbid `scaled(alpha, A)` at compile time, so that users must scale `x` instead

3. Add overloads that take `alpha`, analogous to the rank-1 and rank-k update functions

The first choice is mathematically incorrect, as we will explain below. The second is not incorrect, but could only catch some errors. The third is likewise not incorrect, but would add a lot of overloads for an uncommon use case, and would still not prevent users from scaling the matrix in mathematically incorrect ways.

4.3.4.3.1 Bad choice: Special cases for scaling the matrix

"Treat `scaled(alpha, A)` as a special case" actually means three special cases, given some nested accessor type `Acc` and a scaling factor `alpha` of type `Scalar`.

a. `scaled(alpha, A)`, whose accessor type is `scaled_accessor<Scalar, Acc>`

b. `conjugated(scaled(alpha, A))`, whose accessor type is `conjugated_accessor<scaled_accessor<Scalar, Acc>>`

c. `scaled(alpha, conjugated(A))`, whose accessor type is `scaled_accessor<Scalar, conjugated_accessor<Acc>>`

One could replace `conjugated` with `conjugate_transposed` (which we expect to be more common in practice) without changing the accessor type.

This approach violates the fundamental [linalg] principle that "… each `mdspan` parameter of a function behaves as itself and is not otherwise 'modified' by other parameters" (Section 10.2.5, P1673R13). The behavior of [linalg] is agnostic of specific accessor types, even though implementations likely have optimizations for specific accessor types. [linalg] takes this approach for consistency, even where it results in different behavior from the BLAS (see Section 10.5.2 of P1673R13). The application of this principle here is "the input parameter `A` is always Hermitian."

In this case, the consistency matters for mathematical correctness. What if `scaled(alpha, A)` is Hermitian, but `A` itself is not? An example is $\alpha = -i$ and A is the 2 x 2 matrix whose elements are all i. If we treat `scaled_accessor` as a special case, then `hermitian_matrix_vector_product` will compute different results.

Another problem is that users are permitted to define their own accessor types, including nested accessors. Arbitrary user accessors might have `scaled_accessor` somewhere in that nesting, or they might have the *effect* of `scaled_accessor` without being called that. Thus, we can't detect all possible ways that the matrix might be scaled.

4.3.4.3.2 Not-so-good choice: Forbid scaling the matrix at compile time

Hermitian matrix-matrix and matrix-vector product functions could instead *forbid* scaling the matrix at compile time, at least for the three accessor type cases listed in the previous option.

a. `scaled_accessor<Scalar, Acc>`

b. `conjugated_accessor<scaled_accessor<Scalar, Acc>>`

c. `scaled_accessor<Scalar, conjugated_accessor<Acc>>`

Doing this would certainly encourage correct behavior for the most common cases. However, as mentioned above, users are permitted to define their own accessor types, including nested accessors. Thus, we can't detect all ways that an arbitrary, possibly user-defined accessor might scale the matrix. Furthermore, scaling the matrix might still be mathematically correct. A scaling factor with nonzero imaginary part might even *make* the matrix Hermitian. Applying i as a scaling factor twice would give a perfectly valid scaling factor of -1.

4.3.4.3.3 Not-so-good choice: Add alpha overloads

One could imagine adding overloads that take `alpha`, analogous to the rank-1 and rank-k update overloads that take `alpha`. Users could then write

```
        hermitian_matrix_vector_product(alpha, A, upper_triangle, x, y);
```

instead of

```
        hermitian_matrix_vector_product(A, upper_triangle, scaled(alpha,
x), y);
```

We do not support this approach. First, it would introduce many overloads, without adding to what the existing overloads can accomplish. (Users can already introduce the scaling factor `alpha` through `x`.) Contrast this with the rank-1 and rank-k Hermitian update functions, where not having `alpha` overloads might break simple cases. Here are two examples.

1. If the matrix and vector element types and the scaling factor $\alpha = 2$ are all integers, then the update $C := C - 2xx^H$ can be computed using integer types and arithmetic with an `alpha` overload. However, without an `alpha` overload, the user would need to use `scaled(sqrt(2), x)` as the input vector, thus forcing use of floating-point arithmetic that may give inexact results.

2. The update $C := C - xx^H$ would require resorting to complex arithmetic, as the only way to express $-xx^H$ with the same scaling factor for both vectors is $(ix)(ix)^H$.

Second, `alpha` overloads would not prevent users from *also* supplying `scaled(gamma, A)` as the matrix for some other scaling factor `gamma`. Thus, instead of solving the problem, the overloads would introduce more possibilities for errors.

§ 4.3.5 What if `Scalar` is noncomplex but `conj` is ADL-findable?

Our proposed change defines a "noncomplex number" at compile time. We say that complex numbers have `conj` that is findable by ADL, and noncomplex numbers are either arithmetic types or do not have an ADL-findable `conj`. We choose this definition because it is the same one that we use to define the behavior of `conjugated_accessor` (and also `conjugated`, if P3050 is adopted). It also is the C++ analog of what the BLAS does, namely specify the type of the `alpha` argument as real instead of complex.

This definition is conservative, because it excludes complex numbers with zero imaginary part. For `conjugated_accessor` and `conjugated`, this does not matter; the class and function behave the same from the user's perspective. The exposition-only function *`conj_if_needed`* specifically exists so that `conjugated_accessor` and `conjugated` do not change their input `mdspan`'s `value_type`. However, for the rank-1 and rank-k Hermitian update functions affected by this proposal, constraining `Scalar alpha` at compile time to be noncomplex prevents users from calling those functions with a "complex" number `alpha` whose imaginary part is zero.

This matters if the user defines a number type `Real` that is meant to represent noncomplex numbers, but nevertheless has an ADL-findable `conj`, thus making it a "complex" number type from the perspective of [linalg] functions. There are two ways users might define `conj(Real)`.

1. *Imitating* `std::complex`: Users might define a complex number type `UserComplex` whose real and imaginary parts have type `Real`, and then imitate the behavior of `std::conj(double)` by defining `UserComplex conj(Real x)` to return a `UserComplex` number with real part `x` and imaginary part zero.

2. *Type-preserving*: `Real conj(Real x)` returns `x`.

Option (1) would be an unfortunate choice. [linalg] defines *`conj_if_needed`* specifically to fix the problem that `std::conj(double)` returns `std::complex<double>` instead of `double`. However, Option (2) would be a reasonable thing for users to do, especially if they have designed custom number types without [linalg] in mind. One could accommodate such users by relaxing the constraint on `Scalar` and taking one of the following two approaches.

1. Adding a precondition that *`imag_if_needed`*`(alpha)` equals `Scalar{}`

2. Imitating LWG 4136, by defining the scaling factor to be *real-if-needed*(alpha) instead of alpha

We did not take Approach (1), because adding a precondition decreases safety by adding undefined behavior. It also forces users to add run-time checks. Defining those checks correctly for generic, possibly but not necessarily complex number types would be challenging. We did not take Approach (2) because its behavior would deviate from the BLAS, which requires the scaling factor alpha to be noncomplex at compile time.

§ 4.4 Triangular matrix products, unit diagonals, and scaling factors

1. In BLAS, triangular matrix-vector and matrix-matrix products apply alpha scaling to the implicit unit diagonal. In [linalg], the scaling factor alpha is not applied to the implicit unit diagonal. This is because the library does not interpret scaled(alpha, A) differently than any other mdspan.

2. Users of triangular matrix-vector products can recover BLAS functionality by scaling the input vector instead of the input matrix, so this only matters for triangular matrix-matrix products.

3. All calls of the BLAS's triangular matrix-matrix product routine xTRMM in LAPACK (other than in testing routines) use alpha equal to one.

4. Straightforward approaches for fixing this issue would not break backwards compatibility.

5. Therefore, we do not consider fixing this a high-priority issue, and we do not propose a fix for it in this paper.

§ 4.4.1 BLAS applies alpha after unit diagonal; linalg applies it before

The triangular_matrix_vector_product and triangular_matrix_product algorithms have an implicit_unit_diagonal option. This makes the algorithm not access the diagonal of the matrix, and compute as if the diagonal were all ones. The option corresponds to the BLAS's "Unit" flag. BLAS routines that take both a "Unit" flag and an alpha scaling factor apply "Unit" *before* scaling by alpha, so that the matrix is treated as if it has a diagonal of all alpha values. In contrast, [linalg] follows the general principle that scaled(alpha, A) should be treated like any other kind of mdspan. As a result, algorithms interpret implicit_unit_diagonal as applied to the matrix *after* scaling by alpha, so that the matrix still has a diagonal of all ones.

§ 4.4.2 Triangular solve algorithms not affected

The triangular solve algorithms in std::linalg are not affected, because their BLAS analogs either do not take an alpha argument (as with xTRSV), or the alpha argument does not

affect the triangular matrix (with `xTRSM`, `alpha` affects the right-hand sides `B`, not the triangular matrix `A`).

§ 4.4.3 Triangular matrix-vector product work-around

This issue only reduces functionality of `triangular_matrix_product`. Users of `triangular_matrix_vector_product` who wish to replicate the original BLAS functionality can scale the input matrix (by supplying `scaled(alpha, x)` instead of `x` as the input argument) instead of the triangular matrix.

§ 4.4.4 Triangular matrix-matrix product example

The following example computes $A := 2AB$ where A is a lower triangular matrix, but it makes the diagonal of A all ones on the input (right-hand) side.

```
        triangular_matrix_product(scaled(2.0, A), lower_triangle,
implicit_unit_diagonal, B, A);
```

Contrast with the analogous BLAS routine `DTRMM`, which has the effect of making the diagonal elements all `2.0`.

```
        dtrmm('Left side', 'Lower triangular', 'No transpose', 'Unit
diagonal', m, n, 2.0, A, lda, B, ldb)
```

If we want to use [linalg] to express what the BLAS call expresses, we need to perform a separate scaling.

```
        triangular_matrix_product(A, lower_triangle,
implicit_unit_diagonal, B, A);
        scale(2.0, A);
```

This is counterintuitive, and may also affect performance. Performance of `scale` is typically bound by memory bandwidth and/or latency, but if the work done by `scale` could be fused with the work done by the `triangular_matrix_product`, then `scale`'s memory operations could be "hidden" in the cost of the matrix product.

§ 4.4.5 LAPACK never calls `xTRMM` with the implicit unit diagonal option and `alpha` not equal to one

How much might users care about this missing [linalg] feature? P1673R13 explains that the BLAS was codesigned with LAPACK and that every reference BLAS routine is used by some LAPACK routine. "The BLAS does not aim to provide a complete set of mathematical operations. Every function in the BLAS exists because some LINPACK or LAPACK algorithm needs it" (Section 10.6.1). Therefore, to judge the urgency of adding new functionality to [linalg], we can ask whether the functionality would be needed by a C++ re-implementation of LAPACK. We think not much, because the highest-priority target audience of the BLAS is LAPACK developers, and LAPACK routines (other than testing routines) never use a scaling factor alpha other than one.

We survey calls to `xTRMM` in the latest version of LAPACK as of the publication date of R1 of this proposal, LAPACK 3.12.0. It suffices to survey `DTRMM`, the double-precision real case, since for all the routines of interest, the complex case follows the same pattern. (We did survey `ZTRMM`, the double-precision complex case, just in case.) LAPACK has 24 routines that call `DTRMM` directly. They fall into five categories.

1. Test routines: `DCHK3`, `DCHKE`, `DLARHS`

2. Routines relating to QR factorization or using the result of a QR factorization (especially with block Householder reflectors): `DGELQT3`, `DLARFB`, `DGEQRT3`, `DLARFB_GETT`, `DLARZB`, `DORM22`

3. Routines relating to computing an inverse of a triangular matrix or of a matrix that has been factored into triangular matrices: `DLAUUM`, `DTRITRI`, `DTFTRI`, `DPFTRI`

4. Routines relating to solving eigenvalue (or generalized eigenvalue) problems: `DLAHR2`, `DSYGST`, `DGEHRD`, `DSYGV`, `DSYGV_2STAGE`, `DSYGVD`, `DSYGVX` (note that `DLAQR5` depends on `DTRMM` via `EXTERNAL` declaration, but doesn't actually call it)

5. Routines relating to symmetric indefinite factorizations: `DSYT01_AA`, `DSYTRI2X`, `DSYTRI_3X`

The only routines that call `DTRMM` with `alpha` equal to anything other than one or negative one are the testing routines. Some calls in `DGELQT3` and `DLARFB_GETT` use negative one, but these calls never specify an implicit unit diagonal (they use the explicit diagonal option). The only routine that might possibly call `DTRMM` with both negative one as alpha and the implicit unit diagonal is `DTFTRI`. (This routine "computes the inverse of a triangular matrix A stored in RFP [Rectangular Full Packed] format." RFP format was introduced to LAPACK in the late 2000's, well after the BLAS Standard was published. See LAPACK Working Note 199, which was published in 2008.) `DTFTRI` passes its caller's `diag` argument (which specifies either implicit unit diagonal or explicit diagonal) to `DTRMM`. The only two LAPACK routines that call `DTFTRI` are `DERRRFP` (a testing routine) and `DPFTRI`. `DPFTRI` only ever calls `DTFTRI` with `diag` *not* specifying the implicit unit diagonal option. Therefore, LAPACK never needs both `alpha` not equal to one and the implicit unit diagonal option, so adding the ability to "scale the implicit diagonal" in [linalg] is a low-priority feature.

§ 4.4.6 Fixes would not break backwards compatibility

We can think of two ways to fix this issue. First, we could add an `alpha` scaling parameter, analogous to the symmetric and Hermitian rank-1 and rank-k update functions. Second, we could add a new kind of `Diagonal` template parameter type that expresses a "diagonal value." For example, `implicit_diagonal_t{alpha}` (or a function form, `implicit_diagonal(alpha)`) would tell the algorithm not to access the diagonal elements, but instead to assume that their value is `alpha`. Both of these solutions would let users specify the diagonal's scaling factor separately from the scaling factor for the rest of the matrix. Those two scaling factors could differ, which is new functionality not offered

by the BLAS. More importantly, both of these solutions could be added later, after C++26, without breaking backwards compatibility.

§ 4.5 Triangular solves, unit diagonals, and scaling factors

1. In BLAS, triangular solves with possibly multiple right-hand sides (`xTRSM`) apply `alpha` scaling to the implicit unit diagonal. In [linalg], the scaling factor `alpha` is not applied to the implicit unit diagonal. This is because the library does not interpret `scaled(alpha, A)` differently than any other `mdspan`.

2. Users of triangular solves would need a separate `scale` call to recover BLAS functionality.

3. LAPACK sometimes calls `xTRSM` with `alpha` not equal to one.

4. Straightforward approaches for fixing this issue would not break backwards compatibility.

5. Therefore, we do not consider fixing this a high-priority issue, and we do not propose a fix for it in this paper.

§ 4.5.1 BLAS applies alpha after unit diagonal; linalg applies it before

Triangular solves have a similar issue to the one explained in the previous section. The BLAS routine `xTRSM` applies `alpha` "after" the implicit unit diagonal, while std::linalg applies `alpha` "before." (`xTRSV` does not take an `alpha` scaling factor.) As a result, the BLAS solves with a different matrix than std::linalg.

In mathematical terms, `xTRSM` solves the equation $\alpha(A+I)X = B$ for X, where A is the user's input matrix (without implicit unit diagonal) and I is the identity matrix (with ones on the diagonal and zeros everywhere else). `triangular_matrix_matrix_left_solve` solves the equation $(\alpha A+I)Y = B$ for Y. The two results X and Y are not equal in general.

§ 4.5.2 Work-around requires changing all elements of the matrix

Users could work around this problem by first scaling the matrix A by α, and then solving for Y. In the common case where the "other triangle" of A holds another triangular matrix, users could not call `scale(alpha, A)`. They would instead need to iterate over the elements of A manually. Users might also need to "unscale" the matrix after the solve. Another option would be to copy the matrix A before scaling.

```
        for (size_t i = 0; i < A.extent(0); ++i) {
          for (size_t j = i + 1; j < A.extent(1); ++j) {
            A[i, j] *= alpha;
          }
        }
        triangular_matrix_matrix_left_solve(A, lower_triangle,
implicit_unit_diagonal, B, Y);
```

```
        for (size_t i = 0; i < A.extent(0); ++i) {
          for (size_t j = i + 1; j < A.extent(1); ++j) {
            A[i, j] /= alpha;
          }
        }
```

Users cannot solve this problem by scaling B (either with `scaled(1.0 / alpha, B)` or with `scale(1.0 / alpha, B)`). Transforming X into Y or vice versa is mathematically nontrivial in general, and may introduce new failure conditions. This issue occurs with both the in-place and out-of-place triangular solves.

§ 4.5.3 Unsupported case occurs in LAPACK

The common case in LAPACK is calling `xTRSM` with `alpha` equal to one, but other values of `alpha` occur. For example, `xTRTRI` calls `xTRSM` with `alpha` equal to -1. Thus, we cannot dismiss this issue, as we could with `xTRMM`.

§ 4.5.4 Fixes would not break backwards compatibility

As with triangular matrix products above, we can think of two ways to fix this issue. First, we could add an `alpha` scaling parameter, analogous to the symmetric and Hermitian rank-1 and rank-k update functions. Second, we could add a new kind of `Diagonal` template parameter type that expresses a "diagonal value." For example, `implicit_diagonal_t{alpha}` (or a function form, `implicit_diagonal(alpha)`) would tell the algorithm not to access the diagonal elements, but instead to assume that their value is `alpha`. Both of these solutions would let users specify the diagonal's scaling factor separately from the scaling factor for the rest of the matrix. Those two scaling factors could differ, which is new functionality not offered by the BLAS. More importantly, both of these solutions could be added later, after C++26, without breaking backwards compatibility.

§ 5 Ordering with respect to other proposals and LWG issues

We currently have two other `std::linalg` fix papers in review.

— P3222: Fix C++26 by adding `transposed` special cases for P2642 layouts (forwarded by LEWG to LWG on 2024-08-27 pending electronic poll results)

— P3050: "Fix C++26 by optimizing `linalg::conjugated` for noncomplex value types" (forwarded by LEWG to LWG on 2024-09-03 pending electronic poll results)

LEWG was aware of these two papers and this pending paper P3371 in its 2024-09-03 review of P3050R2. All three of these papers increment the value of the `__cpp_lib_linalg` macro. While this technically causes a conflict between the papers, advice from LEWG on 2024-09-03 was not to introduce special wording changes to avoid this conflict.

We also have two outstanding LWG issues.

— LWG4136 specifies the behavior of Hermitian algorithms on diagonal matrix elements with nonzero imaginary part. (As the BLAS Standard specifies and the Reference BLAS implements, the Hermitian algorithms do not access the imaginary parts of diagonal elements, and assume they are zero.) In our view, P3371 does not conflict with LWG4136.

— LWG4137, "Fix Mandates, Preconditions, and Complexity elements of [linalg] algorithms," affects several sections touched by this proposal, including [linalg.algs.blas3.rankk] and [linalg.algs.blas3.rank2k]. We consider P3371 rebased atop the wording changes proposed by LWG4137. While the wording changes may conflict in a formal ("diff") sense, it is our view that they do not conflict in a mathematical or specification sense.

§ 6 Acknowledgments

Many thanks (with permission) to Raffaele Solcà (CSCS Swiss National Supercomputing Centre, `raffaele.solca@cscs.ch`) for pointing out some of the issues fixed by this paper, as well as the issues leading to LWG4137.

§ 7 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The 🟩 character is used to denote a placeholder section number which the editor shall determine.

In **[version.syn]**, for the following definition,

```
#define __cpp_lib_linalg YYYYMML // also in <linalg>
```

adjust the placeholder value `YYYYMML` as needed so as to denote this proposal's date of adoption.

§ 7.1 New exposition-only concepts for possibly-packed input and output matrices

Add the following lines to the header `<linalg>` synopsis **[linalg.syn]**, just after the declaration of the exposition-only concept *`inout-matrix`* and before the declaration of the exposition-only concept *`possibly-packed-inout-matrix`*. *[Editorial Note:* This addition is not shown in green, becuase the authors could not convince Markdown to format the code correctly. – *end note]*

```
          template<class T>
            concept possibly-packed-in-matrix = see below;     //
exposition only
```

```
          template<class T>
            concept possibly-packed-out-matrix = see below;     //
exposition only
```

Then, remove the declaration of the exposition-only concept *possibly-packed-inout-matrix* from the header <linalg> synopsis **[linalg.syn]**.

Then, add the following lines to **[linalg.helpers.concepts]**, just after the definition of the exposition-only variable template *is-layout_blas_packed* and just before the definition of the exposition-only concept *possibly-packed-inout-matrix*. *[Editorial Note:* This addition is not shown in green, becuase the authors could not convince Markdown to format the code correctly. – *end note]*

```
          template<class T>
            concept possibly-packed-in-matrix =
              is-mdspan<T> && T::rank() == 2 &&
              (T::is_always_unique() || is-layout-blas-packed<typename
T::layout_type>);

          template<class T>
            concept possibly-packed-out-matrix =
              is-mdspan<T> && T::rank() == 2 &&
              is_assignable_v<typename T::reference, typename
T::element_type> &&
              (T::is_always_unique() || is-layout-blas-packed<typename
T::layout_type>);
```

Then, remove the definition of the exposition-only concept *possibly-packed-inout-matrix* from **[linalg.helpers.concepts]**.

§ 7.2 New exposition-only concept for noncomplex numbers

In the Header <linalg> synopsis [linalg.syn], at the end of the section started by the following comment:

```
// [linalg.helpers.concepts], linear algebra argument concepts,
```

add the following declaration of the exposition-only concept *noncomplex*. *[Editorial Note:* This addition is not shown in green, becuase the authors could not convince Markdown to format the code correctly. – *end note]*

```
        template<class T>
          concept noncomplex = see below; // exposition only
```

In [linalg.helpers.concepts], change paragraph 3 to read as follows (new content in green).

Unless explicitly permitted, any *inout-vector*, *inout-matrix*, *inout-object*, *out-vector*, *out-matrix*, *out-object*, *possibly-packed-out-matrix*, or *possibly-packed-inout-matrix* parameter of a function in [linalg] shall not overlap any other `mdspan` parameter of the function.

Append the following to the end of [linalg.helpers.concepts]. *[Editorial Note:* These additions are not shown in green, becuase the authors could not convince Markdown to format the code correctly. – *end note]*

```
        template<class T>
          concept noncomplex = see below;
```

4 A type `T` models *noncomplex* if `T` is a linear algebra value type, and either

(4.1) `T` is not an arithmetic type, or

(4.2) the expression `conj(E)` is not valid, with overload resolution performed in a context that includes the declaration `template<class T> T conj(const T&) = delete;`.

§ 7.3 Rank-1 update functions in synopsis

In the header `<linalg>` synopsis **[linalg.syn]**, replace all the declarations of all the `matrix_rank_1_update`, `matrix_rank_1_update_c`, `symmetric_matrix_rank_1_update`, and `hermitian_matrix_rank_1_update` overloads to read as follows. *[Editorial Note:* There are three changes here. First, the existing overloads become "overwriting" overloads. Second, new "updating" overloads are added. Third, the `hermitian_rank_1_update` functions that take an `alpha` parameter now constrain `alpha` to be *noncomplex*.

Changes do not use red or green highlighting, becuase the authors could not convince Markdown to format the code correctly. – *end note]*

```
          // [linalg.algs.blas2.rank1], nonsymmetric rank-1 matrix update

          // overwriting nonsymmetric rank-1 matrix update
          template<in-vector InVec1, in-vector InVec2, out-matrix OutMat>
            void matrix_rank_1_update(InVec1 x, InVec2 y, OutMat A);
          template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2, out-matrix OutMat>
            void matrix_rank_1_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, OutMat A);

          template<in-vector InVec1, in-vector InVec2, out-matrix OutMat>
            void matrix_rank_1_update_c(InVec1 x, InVec2 y, OutMat A);
          template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2, out-matrix OutMat>
            void matrix_rank_1_update_c(ExecutionPolicy&& exec,
                                        InVec1 x, InVec2 y, OutMat A);

          // updating nonsymmetric rank-1 matrix update
```

```
          template<in-vector InVec1, in-vector InVec2, in-matrix InMat,
out-matrix OutMat>
            void matrix_rank_1_update(InVec1 x, InVec2 y, InMat E, OutMat
A);
          template<class ExecutionPolicy, in-vector InVec1, in-matrix
InMat, in-vector InVec2, out-matrix OutMat>
            void matrix_rank_1_update(ExecutionPolicy&& exec,
                                      InVec1 x, InVec2 y, InMat E, OutMat
A);

          template<in-vector InVec1, in-vector InVec2, in-matrix InMat,
out-matrix OutMat>
            void matrix_rank_1_update_c(InVec1 x, InVec2 y, InMat E,
OutMat A);
          template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2, in-matrix InMat, out-matrix OutMat>
            void matrix_rank_1_update_c(ExecutionPolicy&& exec,
                                        InVec1 x, InVec2 y, InMat E,
OutMat A);

          // [linalg.algs.blas2.symherrank1], symmetric or Hermitian rank-
1 matrix update

          // overwriting symmetric rank-1 matrix update
          template<class Scalar, in-vector InVec, possibly-packed-out-
matrix OutMat, class Triangle>
            void symmetric_matrix_rank_1_update(Scalar alpha, InVec x,
OutMat A, Triangle t);
          template<class ExecutionPolicy,
                   class Scalar, in-vector InVec, possibly-packed-out-
matrix OutMat, class Triangle>
            void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                                Scalar alpha, InVec x,
OutMat A, Triangle t);
          template<in-vector InVec, possibly-packed-out-matrix OutMat,
class Triangle>
            void symmetric_matrix_rank_1_update(InVec x, OutMat A,
Triangle t);
          template<class ExecutionPolicy,
                   in-vector InVec, possibly-packed-out-matrix OutMat,
class Triangle>
            void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                                InVec x, OutMat A,
Triangle t);

          // updating symmetric rank-1 matrix update
          template<class Scalar, in-vector InVec, possibly-packed-in-
matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_1_update(Scalar alpha, InVec x,
InMat E, OutMat A, Triangle t);
          template<class ExecutionPolicy,
                   class Scalar, in-vector InVec, possibly-packed-in-
matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                                Scalar alpha, InVec x,
```

```
InMat E, OutMat A, Triangle t);
          template<in-vector InVec, possibly-packed-in-matrix InMat,
possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_1_update(InVec x, InMat E, OutMat
A, Triangle t);
          template<class ExecutionPolicy,
                   in-vector InVec, possibly-packed-in-matrix InMat,
possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                                InVec x, InMat E, OutMat
A, Triangle t);

          // overwriting Hermitian rank-1 matrix update
          template<noncomplex Scalar, in-vector InVec, possibly-packed-
out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_1_update(Scalar alpha, InVec x,
OutMat A, Triangle t);
          template<class ExecutionPolicy,
                   noncomplex Scalar, in-vector InVec, possibly-packed-
out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                                Scalar alpha, InVec x,
OutMat A, Triangle t);
          template<in-vector InVec, possibly-packed-out-matrix OutMat,
class Triangle>
            void hermitian_matrix_rank_1_update(InVec x, OutMat A,
Triangle t);
          template<class ExecutionPolicy,
                   in-vector InVec, possibly-packed-out-matrix OutMat,
class Triangle>
            void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                                InVec x, OutMat A,
Triangle t);

          // updating Hermitian rank-1 matrix update
          template<noncomplex Scalar, in-vector InVec, possibly-packed-in-
matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_1_update(Scalar alpha, InVec x,
InMat E, OutMat A, Triangle t);
          template<class ExecutionPolicy,
                   noncomplex Scalar, in-vector InVec, possibly-packed-in-
matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                                Scalar alpha, InVec x,
InMat E, OutMat A, Triangle t);
          template<in-vector InVec, possibly-packed-in-matrix InMat,
possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_1_update(InVec x, InMat E, OutMat
A, Triangle t);
          template<class ExecutionPolicy,
                   in-vector InVec, possibly-packed-in-matrix InMat,
possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                                InVec x, InMat E, OutMat
A, Triangle t);
```

§ 7.4 Rank-2 update functions in synopsis

In the header `<linalg>` synopsis **[linalg.syn]**, replace all the declarations of all the `symmetric_matrix_rank_2_update` and `hermitian_matrix_rank_2_update` overloads to read as follows. *[Editorial Note:* These additions are not shown in green, becuase the authors could not convince Markdown to format the code correctly. *– end note]*

```
          // [linalg.algs.blas2.rank2], symmetric and Hermitian rank-2
matrix updates

          // overwriting symmetric rank-2 matrix update
          template<in-vector InVec1, in-vector InVec2,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, OutMat
A, Triangle t);
          template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                                InVec1 x, InVec2 y, OutMat
A, Triangle t);

          // updating symmetric rank-2 matrix update
          template<in-vector InVec1, in-vector InVec2,
                   possibly-packed-in-matrix InMat,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, InMat
E, OutMat A, Triangle t);
          template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2,
                   possibly-packed-in-matrix InMat,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                                InVec1 x, InVec2 y, InMat
E, OutMat A, Triangle t);

          // overwriting Hermitian rank-2 matrix update
          template<in-vector InVec1, in-vector InVec2,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, OutMat
A, Triangle t);
          template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                                InVec1 x, InVec2 y, OutMat
A, Triangle t);

          // updating Hermitian rank-2 matrix update
          template<in-vector InVec1, in-vector InVec2,
                   possibly-packed-in-matrix InMat,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, InMat
```

```
E, OutMat A, Triangle t);
          template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2,
                   possibly-packed-in-matrix InMat,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                                InVec1 x, InVec2 y, InMat
E, OutMat A, Triangle t);
```

§ 7.5 Rank-k update functions in synopsis

In the header `<linalg>` synopsis **[linalg.syn]**, replace all the declarations of all the `symmetric_matrix_rank_k_update` and `hermitian_matrix_rank_k_update` overloads to read as follows. *[Editorial Note:* These additions are not shown in green, becuase the authors could not convince Markdown to format the code correctly. *– end note]*

```
          // [linalg.algs.blas3.rankk], rank-k update of a symmetric or
Hermitian matrix

          // overwriting symmetric rank-k matrix update
          template<class Scalar,
                   in-matrix InMat,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void symmetric_matrix_rank_k_update(
              Scalar alpha, InMat A, OutMat C, Triangle t);
          template<class ExecutionPolicy, class Scalar,
                   in-matrix InMat,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void symmetric_matrix_rank_k_update(
              ExecutionPolicy&& exec, Scalar alpha, InMat A, OutMat C,
Triangle t);
          template<in-matrix InMat,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void symmetric_matrix_rank_k_update(
              InMat A, OutMat C, Triangle t);
          template<class ExecutionPolicy,
                   in-matrix InMat,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void symmetric_matrix_rank_k_update(
              ExecutionPolicy&& exec, InMat A, OutMat C, Triangle t);

          // updating symmetric rank-k matrix update
          template<class Scalar,
                   in-matrix InMat1,
                   possibly-packed-in-matrix InMat2,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void symmetric_matrix_rank_k_update(
              Scalar alpha,
```

```
            InMat1 A, InMat2 E, OutMat C, Triangle t);
          template<class ExecutionPolicy, class Scalar,
                   in-matrix InMat1,
                   possibly-packed-in-matrix InMat2,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void symmetric_matrix_rank_k_update(
              ExecutionPolicy&& exec, Scalar alpha,
              InMat1 A, InMat2 E, OutMat C, Triangle t);
          template<in-matrix InMat1,
                   possibly-packed-in-matrix InMat2,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void symmetric_matrix_rank_k_update(
              InMat1 A, InMat2 E, OutMat C, Triangle t);
          template<class ExecutionPolicy,
                   in-matrix InMat1,
                   possibly-packed-in-matrix InMat2,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void symmetric_matrix_rank_k_update(
              ExecutionPolicy&& exec,
              InMat1 A, InMat2 E, OutMat C, Triangle t);

          // overwriting rank-k Hermitian matrix update
          template<noncomplex Scalar,
                   in-matrix InMat,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void hermitian_matrix_rank_k_update(
              Scalar alpha, InMat A, OutMat C, Triangle t);
          template<class ExecutionPolicy, noncomplex Scalar,
                   in-matrix InMat,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void hermitian_matrix_rank_k_update(
              ExecutionPolicy&& exec, Scalar alpha, InMat A, OutMat C,
Triangle t);
          template<in-matrix InMat,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void hermitian_matrix_rank_k_update(
              InMat A, OutMat C, Triangle t);
          template<class ExecutionPolicy,
                   in-matrix InMat,
                   possibly-packed-out-matrix OutMat,
                   class Triangle>
            void hermitian_matrix_rank_k_update(
              ExecutionPolicy&& exec, InMat A, OutMat C, Triangle t);

          // updating rank-k Hermitian matrix update
          template<noncomplex Scalar,
                   in-matrix InMat1,
                   possibly-packed-in-matrix InMat2,
                   possibly-packed-out-matrix OutMat,
```

```
                class Triangle>
          void hermitian_matrix_rank_k_update(
            Scalar alpha,
            InMat1 A, InMat2 E, OutMat C, Triangle t);
        template<class ExecutionPolicy, noncomplex Scalar,
                in-matrix InMat1,
                possibly-packed-in-matrix InMat2,
                possibly-packed-out-matrix OutMat,
                class Triangle>
          void hermitian_matrix_rank_k_update(
            ExecutionPolicy&& exec, Scalar alpha,
            InMat1 A, InMat2 E, OutMat C, Triangle t);
        template<in-matrix InMat1,
                possibly-packed-in-matrix InMat2,
                possibly-packed-out-matrix OutMat,
                class Triangle>
          void hermitian_matrix_rank_k_update(
            InMat1 A, InMat2 E, OutMat C, Triangle t);
        template<class ExecutionPolicy,
                in-matrix InMat1,
                possibly-packed-in-matrix InMat2,
                possibly-packed-out-matrix OutMat,
                class Triangle>
          void hermitian_matrix_rank_k_update(
            ExecutionPolicy&& exec,
            InMat1 A, InMat2 E, OutMat C, Triangle t);
```

§ 7.6 Rank-2k update functions in synopsis

In the header `<linalg>` synopsis **[linalg.syn]**, replace all the declarations of all the `symmetric_matrix_rank_2k_update` and `hermitian_matrix_rank_2k_update` overloads to read as follows. *[Editorial Note:* These additions are not shown in green, becuase the authors could not convince Markdown to format the code correctly. *– end note]*

```
        // [linalg.algs.blas3.rank2k], rank-2k update of a symmetric or
Hermitian matrix

        // overwriting symmetric rank-2k matrix update
        template<in-matrix InMat1, in-matrix InMat2,
                possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B,
OutMat C, Triangle t);
        template<class ExecutionPolicy,
                in-matrix InMat1, in-matrix InMat2,
                possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                               InMat1 A, InMat2 B,
OutMat C, Triangle t);

        // updating symmetric rank-2k matrix update
        template<in-matrix InMat1, in-matrix InMat2,
                possibly-packed-in-matrix InMat3,
                possibly-packed-out-matrix OutMat, class Triangle>
```

```
            void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B,
InMat3 E, OutMat C, Triangle t);
          template<class ExecutionPolicy,
                   in-matrix InMat1, in-matrix InMat2,
                   possibly-packed-in-matrix InMat3,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                                 InMat1 A, InMat2 B,
InMat3 E, OutMat C, Triangle t);

          // overwriting Hermitian rank-2k matrix update
          template<in-matrix InMat1, in-matrix InMat2,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B,
OutMat C, Triangle t);
          template<class ExecutionPolicy,
                   in-matrix InMat1, in-matrix InMat2,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                                 InMat1 A, InMat2 B,
OutMat C, Triangle t);

          // updating Hermitian rank-2k matrix update
          template<in-matrix InMat1, in-matrix InMat2,
                   possibly-packed-in-matrix InMat3,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B,
InMat3 E, OutMat C, Triangle t);
          template<class ExecutionPolicy,
                   in-matrix InMat1, in-matrix InMat2,
                   possibly-packed-in-matrix InMat3,
                   possibly-packed-out-matrix OutMat, class Triangle>
            void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                                 InMat1 A, InMat2 B,
InMat3 E, OutMat C, Triangle t);
```

§ 7.7 Specification of nonsymmetric rank-1 update functions

Replace the entire contents of [linalg.algs.blas2.rank1] with the following.

1 The following elements apply to all functions in [linalg.algs.blas2.rank1].

2 *Mandates*:

(2.1) *possibly-multipliable*<OutMat, InVec2, InVec1>() is true, and

(2.2) *possibly-addable*(A, E, A) is true for those overloads that take an E parameter.

3 *Preconditions*:

(3.1) multipliable(A, y, x) is true, and

(3.2) *addable*(A, E, A) is true for those overloads that take an E parameter.

4 *Complexity*: $O($ x.extent(0) $\times$ y.extent(0) $)$.

```
        template<in-vector InVec1, in-vector InVec2, out-matrix OutMat>
          void matrix_rank_1_update(InVec1 x, InVec2 y, OutMat A);
        template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2, out-matrix OutMat>
          void matrix_rank_1_update(ExecutionPolicy&& exec, InVec1 x,
InVec2 y, OutMat A);
```

5 These functions perform a overwriting nonsymmetric nonconjugated rank-1 update.

[Note: These functions correspond to the BLAS functions xGER (for real element types) and xGERU (for complex element types)[bib]. *– end note]*

6 *Effects*: Computes $A = xy^T$.

```
        template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-
matrix OutMat>
          void matrix_rank_1_update(InVec1 x, InVec2 y, InMat E, OutMat
A);
        template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2, in-matrix InMat, out-matrix OutMat>
          void matrix_rank_1_update(ExecutionPolicy&& exec, InVec1 x,
InVec2 y, InMat E, OutMat A);
```

7 These functions perform an updating nonsymmetric nonconjugated rank-1 update.

[Note: These functions correspond to the BLAS functions xGER (for real element types) and xGERU (for complex element types)[bib]. *– end note]*

8 *Effects*: Computes $A = E + xy^T$.

9 *Remarks*: A may alias E.

```
        template<in-vector InVec1, in-vector InVec2, out-matrix OutMat>
          void matrix_rank_1_update_c(InVec1 x, InVec2 y, OutMat A);
        template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2, out-matrix OutMat>
          void matrix_rank_1_update_c(ExecutionPolicy&& exec, InVec1 x,
InVec2 y, OutMat A);
```

10 These functions perform a overwriting nonsymmetric conjugated rank-1 update.

[Note: These functions correspond to the BLAS functions xGER (for real element types) and xGERC (for complex element types)[bib]. *– end note]*

11 *Effects*:

(11.1) For the overloads without an ExecutionPolicy argument, equivalent to:

```
        matrix_rank_1_update(x, conjugated(y), A);
```

(11.2) otherwise, equivalent to:

```
        matrix_rank_1_update(std::forward<ExecutionPolicy>(exec), x,
conjugated(y), A);
```

§ 7.8 Specification of symmetric and Hermitian rank-1 update functions

Replace the entire contents of [linalg.algs.blas2.symherrank1] with the following.

1 *[Note:* These functions correspond to the BLAS functions `xSYR`, `xSPR`, `xHER`, and `xHPR`[bib]. They have overloads taking a scaling factor `alpha`, because it would be impossible to express the update $A = Axx^T$ otherwise. – *end note]*

2 The following elements apply to all functions in [linalg.algs.blas2.symherrank1].

3 For any function `F` in this section that takes a parameter named `t`, an `InMat` template parameter, and a function parameter `InMat E`, `t` applies to accesses done through the parameter `E`. `F` will only access the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` will use the value *`real-if-needed`*`(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` will use the value

(3.1) *`conj-if-needed`*`(E[j, i])` if the name of `F` starts with `hermitian`, or

(3.2) `E[j, i]` if the name of `F` starts with `symmetric`.

4 *Mandates*:

(4.1) If `OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.2) If the function has an `InMat` template parameter and `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.3) *`compatible-static-extents`*`<decltype(A), decltype(A)>(0, 1)` is `true`;

(4.4) *`compatible-static-extents`*`<decltype(A), decltype(x)>(0, 0)` is `true`; and

(4.5) *`possibly-addable`*`<decltype(A), decltype(E), decltype(A)>` is `true` for those overloads that take an `E` parameter.

5 *Preconditions*:

(5.1) `A.extent(0)` equals `A.extent(1)`,

(5.2) `A.extent(0)` equals `x.extent(0)`, and

(5.3) *addable*(A, E, A) is true for those overloads that take an E parameter.

6 *Complexity*: *O*(x.extent(0) × x.extent(0)).

```
        template<class Scalar, in-vector InVec, possibly-packed-out-matrix
OutMat, class Triangle>
          void symmetric_matrix_rank_1_update(Scalar alpha, InVec x,
OutMat A, Triangle t);
        template<class ExecutionPolicy,
                 class Scalar, in-vector InVec, possibly-packed-out-matrix
OutMat, class Triangle>
          void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                              Scalar alpha, InVec x,
OutMat A, Triangle t);
```

7 These functions perform an overwriting symmetric rank-1 update of the symmetric matrix A, taking into account the Triangle parameter that applies to A ([linalg.general]).

8 *Effects*: Computes $A = \alpha x x^T$, where the scalar α is alpha.

```
        template<in-vector InVec, possibly-packed-out-matrix OutMat, class
Triangle>
          void symmetric_matrix_rank_1_update(InVec x, OutMat A, Triangle
t);
        template<class ExecutionPolicy,
                 in-vector InVec, possibly-packed-out-matrix OutMat, class
Triangle>
          void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
InVec x, OutMat A, Triangle t);
```

9 These functions perform an overwriting symmetric rank-1 update of the symmetric matrix A, taking into account the Triangle parameter that applies to A ([linalg.general]).

10 *Effects*: Computes $A = x x^T$.

```
        template<class Scalar, in-vector InVec, possibly-packed-in-matrix
InMat, possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, InMat
E, OutMat A, Triangle t);
        template<class ExecutionPolicy,
                 class Scalar, in-vector InVec, possibly-packed-in-matrix
InMat, possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                              Scalar alpha, InVec x, InMat
E, OutMat A, Triangle t);
```

11 These functions perform an updating symmetric rank-1 update of the symmetric matrix A using the symmetric matrix E, taking into account the Triangle parameter that applies to A and E ([linalg.general]).

12 *Effects*: Computes $A = E + \alpha x x^T$, where the scalar α is alpha.

```
        template<in-vector InVec, possibly-packed-in-matrix InMat,
possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_1_update(InVec x, InMat E, OutMat A,
Triangle t);
        template<class ExecutionPolicy,
                 in-vector InVec, possibly-packed-in-matrix InMat,
possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                              InVec x, InMat E, OutMat A,
Triangle t);
```

13 These functions perform an updating symmetric rank-1 update of the symmetric matrix `A` using the symmetric matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

14 *Effects*: Computes $A = E + xx^T$.

```
        template<noncomplex Scalar, in-vector InVec, possibly-packed-out-
matrix OutMat, class Triangle>
          void hermitian_matrix_rank_1_update(Scalar alpha, InVec x,
OutMat A, Triangle t);
        template<class ExecutionPolicy,
                 noncomplex Scalar, in-vector InVec, possibly-packed-out-
matrix OutMat, class Triangle>
          void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                              Scalar alpha, InVec x,
OutMat A, Triangle t);
```

15 These functions perform an overwriting Hermitian rank-1 update of the Hermitian matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

16 *Effects*: Computes $A = \alpha xx^H$, where the scalar α is `alpha`.

```
        template<in-vector InVec, possibly-packed-out-matrix OutMat, class
Triangle>
          void hermitian_matrix_rank_1_update(InVec x, OutMat A, Triangle
t);
        template<class ExecutionPolicy,
                 in-vector InVec, possibly-packed-out-matrix OutMat, class
Triangle>
          void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
InVec x, OutMat A, Triangle t);
```

17 These functions perform an overwriting Hermitian rank-1 update of the Hermitian matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

18 *Effects*: Computes $A = xx^T$.

```
        template<noncomplex Scalar, in-vector InVec, possibly-packed-in-
matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
          void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, InMat
E, OutMat A, Triangle t);
        template<class ExecutionPolicy,
```

```
                  noncomplex Scalar, in-vector InVec, possibly-packed-in-
matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
          void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                              Scalar alpha, InVec x, InMat
E, OutMat A, Triangle t);
```

19 These functions perform an updating Hermitian rank-1 update of the Hermitian matrix `A` using the Hermitian matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

20 *Effects*: Computes $A = E + \alpha x x^H$, where the scalar α is `alpha`.

```
        template<in-vector InVec, possibly-packed-in-matrix InMat,
possibly-packed-out-matrix OutMat, class Triangle>
          void hermitian_matrix_rank_1_update(InVec x, InMat E, OutMat A,
Triangle t);
        template<class ExecutionPolicy,
                 in-vector InVec, possibly-packed-in-matrix InMat,
possibly-packed-out-matrix OutMat, class Triangle>
          void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                              InVec x, InMat E, OutMat A,
Triangle t);
```

21 These functions perform an updating Hermitian rank-1 update of the Hermitian matrix `A` using the Hermitian matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

22 *Effects*: Computes $A = E + xx^T$.

§ 7.9 Specification of symmetric and Hermitian rank-2 update functions

Replace the entire contents of [linalg.algs.blas2.rank2] with the following.

1 *[Note:* These functions correspond to the BLAS functions `xSYR2`, `xSPR2`, `xHER2`, and `xHPR2` [bib]. *– end note]*

2 The following elements apply to all functions in [linalg.algs.blas2.rank2].

3 For any function `F` in this section that takes a parameter named `t`, an `InMat` template parameter, and a function parameter `InMat E`, `t` applies to accesses done through the parameter `E`. `F` will only access the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` will use the value *`real-if-needed`*`(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` will use the value

(3.1) *`conj-if-needed`*`(E[j, i])` if the name of `F` starts with `hermitian`, or

(3.2) `E[j, i]` if the name of `F` starts with `symmetric`.

4 *Mandates*:

(4.1) If `OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.2) If the function has an `InMat` template parameter and `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(4.3) *`compatible-static-extents`*`<decltype(A), decltype(A)>(0, 1)` is `true`;

(4.4) *`possibly-multipliable`*`<decltype(A), decltype(x), decltype(y)>()` is `true`; and

(4.5) *`possibly-addable`*`<decltype(A), decltype(E), decltype(A)>` is `true` for those overloads that take an `E` parameter.

5 *Preconditions*:

(5.1) `A.extent(0)` equals `A.extent(1)`,

(5.2) *`multipliable`*`(A, x, y)` is `true`, and

(5.3) *`addable`*`(A, E, A)` is `true` for those overloads that take an `E` parameter.

6 *Complexity*: *O*(`x.extent(0)` × `y.extent(0)`).

```
        template<in-vector InVec1, in-vector InVec2,
                 possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, OutMat
A, Triangle t);
        template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2,
                 possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                              InVec1 x, InVec2 y, OutMat
A, Triangle t);
```

7 These functions perform an overwriting symmetric rank-2 update of the symmetric matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

8 Effects: Computes $A = xy^T + yx^T$.

```
        template<in-vector InVec1, in-vector InVec2,
                 possibly-packed-in-matrix InMat,
                 possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E,
OutMat A, Triangle t);
        template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2,
                 possibly-packed-in-matrix InMat,
                 possibly-packed-out-matrix OutMat, class Triangle>
          void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
```

```
                                                InVec1 x, InVec2 y, InMat E,
OutMat A, Triangle t);
```

9 These functions perform an updating symmetric rank-2 update of the symmetric matrix `A` using the symmetric matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

10 Effects: Computes $A = E + xy^T + yx^T$.

```
        template<in-vector InVec1, in-vector InVec2,
                 possibly-packed-out-matrix OutMat, class Triangle>
          void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, OutMat
A, Triangle t);
        template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2,
                 possibly-packed-out-matrix OutMat, class Triangle>
          void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                              InVec1 x, InVec2 y, OutMat
A, Triangle t);
```

11 These functions perform an overwriting Hermitian rank-2 update of the Hermitian matrix `A`, taking into account the `Triangle` parameter that applies to `A` ([linalg.general]).

12 Effects: Computes $A = xy^H + yx^H$.

```
        template<in-vector InVec1, in-vector InVec2,
                 possibly-packed-in-matrix InMat,
                 possibly-packed-out-matrix OutMat, class Triangle>
          void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E,
OutMat A, Triangle t);
        template<class ExecutionPolicy, in-vector InVec1, in-vector
InVec2,
                 possibly-packed-in-matrix InMat,
                 possibly-packed-out-matrix OutMat, class Triangle>
          void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                              InVec1 x, InVec2 y, InMat E,
OutMat A, Triangle t);
```

13 These functions perform an updating Hermitian rank-2 update of the Hermitian matrix `A` using the Hermitian matrix `E`, taking into account the `Triangle` parameter that applies to `A` and `E` ([linalg.general]).

14 Effects: Computes $A = E + xy^H + yx^H$.

§ 7.10 Specification of rank-k update functions

Replace the entire contents of [linalg.algs.blas3.rankk] with the following.

[Note: These functions correspond to the BLAS functions `xSYRK` and `xHERK`. – *end note]*

1 The following elements apply to all functions in [linalg.algs.blas3.rankk].

2 For any function `F` in this section that takes a parameter named `t`, an `InMat2` template parameter, and a function parameter `InMat2 E`, `t` applies to accesses done through the parameter `E`. `F` will only access the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` will use the value *`real-if-needed`*`(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` will use the value

(2.1) *`conj-if-needed`*`(E[j, i])` if the name of `F` starts with `hermitian`, or

(2.2) `E[j, i]` if the name of `F` starts with `symmetric`.

3 *Mandates:*

(3.1) — If `OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument.

(3.2) — If the function takes an `InMat2` template parameter and if `InMat2` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument.

(3.3) — *`possibly-multipliable`*`<decltype(A), decltype(transposed(A)), decltype(C)>` is `true`.

(3.4) — *`possibly-addable`*`<decltype(C), decltype(E), decltype(C)>` is `true` for those overloads that take an `E` parameter.

4 *Preconditions:*

(4.1) — *`multipliable`*`(A, transposed(A), C)` is `true`. *[Note:* This implies that `C` is square. *– end note]*

(4.2) — *`addable`*`(C, E, C)` is `true` for those overloads that take an `E` parameter.

5 *Complexity: O(* `A.extent(0)` · `A.extent(1)` · `A.extent(0)`).

6 *Remarks:* `C` may alias `E` for those overloads that take an `E` parameter.

```
template<class Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  Scalar alpha,
  InMat A,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
```

```
  ExecutionPolicy&& exec,
  Scalar alpha,
  InMat A,
  OutMat C,
  Triangle t);
```

5 *Effects:* Computes $C = \alpha AA^T$, where the scalar α is `alpha`.

```
template<in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  InMat A,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
  ExecutionPolicy&& exec,
  InMat A,
  OutMat C,
  Triangle t);
```

6 *Effects:* Computes $C = AA^T$.

```
template<noncomplex Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  Scalar alpha,
  InMat A,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         noncomplex Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  ExecutionPolicy&& exec,
  Scalar alpha,
  InMat A,
  OutMat C,
  Triangle t);
```

7 *Effects:* Computes $C = \alpha AA^H$, where the scalar α is `alpha`.

```
template<in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
  InMat A,
```

```
    OutMat C,
    Triangle t);
  template<class ExecutionPolicy,
           in-matrix InMat,
           possibly-packed-out-matrix OutMat,
           class Triangle>
  void hermitian_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    InMat A,
    OutMat C,
    Triangle t);
```

8 *Effects:* Computes $C = AA^H$.

```
  template<class Scalar,
           in-matrix InMat1,
           possibly-packed-in-matrix InMat2,
           possibly-packed-out-matrix OutMat,
           class Triangle>
  void symmetric_matrix_rank_k_update(
    Scalar alpha,
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);
  template<class ExecutionPolicy,
           class Scalar,
           in-matrix InMat1,
           possibly-packed-in-matrix InMat2,
           possibly-packed-out-matrix OutMat,
           class Triangle>
  void symmetric_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    Scalar alpha,
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);
```

9 *Effects:* Computes $C = E + \alpha AA^T$, where the scalar α is `alpha`.

```
  template<in-matrix InMat1,
           possibly-packed-in-matrix InMat2,
           possibly-packed-out-matrix OutMat,
           class Triangle>
  void symmetric_matrix_rank_k_update(
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);
  template<class ExecutionPolicy,
           in-matrix InMat1,
           possibly-packed-in-matrix InMat2,
           possibly-packed-out-matrix OutMat,
           class Triangle>
```

```
        void symmetric_matrix_rank_k_update(
          ExecutionPolicy&& exec,
          InMat1 A,
          InMat2 E,
          OutMat C,
          Triangle t);
```

10 *Effects:* Computes $C = E + AA^T$.

```
        template<noncomplex Scalar,
                 in-matrix InMat1,
                 possibly-packed-in-matrix InMat2,
                 possibly-packed-out-matrix OutMat,
                 class Triangle>
        void hermitian_matrix_rank_k_update(
          Scalar alpha,
          InMat1 A,
          InMat2 E,
          OutMat C,
          Triangle t);
        template<class ExecutionPolicy,
                 noncomplex Scalar,
                 in-matrix InMat1,
                 possibly-packed-in-matrix InMat2,
                 possibly-packed-out-matrix OutMat,
                 class Triangle>
        void hermitian_matrix_rank_k_update(
          ExecutionPolicy&& exec,
          Scalar alpha,
          InMat1 A,
          InMat2 E,
          OutMat C,
          Triangle t);
```

11 *Effects:* Computes $C = E + \alpha AA^H$, where the scalar α is `alpha`.

```
        template<in-matrix InMat1,
                 possibly-packed-in-matrix InMat2,
                 possibly-packed-out-matrix OutMat,
                 class Triangle>
        void hermitian_matrix_rank_k_update(
          InMat1 A,
          InMat2 E,
          OutMat C,
          Triangle t);
        template<class ExecutionPolicy,
                 in-matrix InMat1,
                 possibly-packed-in-matrix InMat2,
                 possibly-packed-out-matrix OutMat,
                 class Triangle>
        void hermitian_matrix_rank_k_update(
          ExecutionPolicy&& exec,
          InMat1 A,
          InMat2 E,
```

```
          OutMat C,
          Triangle t);
```

12 *Effects:* Computes $C = E + AA^H$.

§ 7.11 Specification of rank-2k update functions

Replace the entire contents of [linalg.algs.blas3.rank2k] with the following.

[Note: These functions correspond to the BLAS functions `xSYR2K` and `xHER2K`. – *end note]*

1 The following elements apply to all functions in [linalg.algs.blas3.rank2k].

2 For any function `F` in this section that takes a parameter named `t`, an `InMat3` template parameter, and a function parameter `InMat3 E`, `t` applies to accesses done through the parameter `E`. `F` will only access the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` will use the value *`real-if-needed`*`(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` will use the value

(2.1) *`conj-if-needed`*`(E[j, i])` if the name of `F` starts with `hermitian`, or

(2.2) `E[j, i]` if the name of `F` starts with `symmetric`.

3 *Mandates:*

(3.1) — If `OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;

(3.2) — If the function takes an `InMat3` template parameter and if `InMat3` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument.

(3.3) — *`possibly-multipliable`*`<decltype(A), decltype(transposed(B)), decltype(C)>` is `true`.

(3.4) — *`possibly-multipliable`*`<decltype(B), decltype(transposed(A)), decltype(C)>` is `true`.

(3.5) — *`possibly-addable`*`<decltype(C), decltype(E), decltype(C)>` is `true` for those overloads that take an `E` parameter.

4 *Preconditions:*

(4.1) — *`multipliable`*`(A, transposed(B), C)` is `true`.

(4.2) — *`multipliable`*`(B, transposed(A), C)` is `true`. *[Note:* This and the previous imply that `C` is square. – *end note]*

(4.3) — *`addable`*`(C, E, C)` is `true` for those overloads that take an `E` parameter.

4 *Complexity:* $O($ `A.extent(0)` $\cdot$ `A.extent(1)` $\cdot$ `B.extent(0)` $)$

5 *Remarks:* `C` may alias `E` for those overloads that take an `E` parameter.

```
template<in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_2k_update(
  InMat1 A,
  InMat2 B,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_2k_update(
  ExecutionPolicy&& exec,
  InMat1 A,
  InMat2 B,
  OutMat C,
  Triangle t);
```

5 *Effects:* Computes $C = AB^T + BA^T$.

```
template<in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_2k_update(
  InMat1 A,
  InMat2 B,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_2k_update(
  ExecutionPolicy&& exec,
  InMat1 A,
  InMat2 B,
  OutMat C,
  Triangle t);
```

6 *Effects:* Computes $C = AB^H + BA^H$.

```
template<in-matrix InMat1,
         in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat,
         class Triangle>
```

```
void symmetric_matrix_rank_2k_update(
  InMat1 A,
  InMat2 B,
  InMat3 E,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1,
         in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_2k_update(
  ExecutionPolicy&& exec,
  InMat1 A,
  InMat2 B,
  InMat3 E,
  OutMat C,
  Triangle t);
```

7 *Effects:* Computes $C = E + AB^T + BA^T$.

```
template<in-matrix InMat1,
         in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_2k_update(
  InMat1 A,
  InMat2 B,
  InMat3 E,
  OutMat C,
  Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat1,
         in-matrix InMat2,
         in-matrix InMat3,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_2k_update(
  ExecutionPolicy&& exec,
  InMat1 A,
  InMat2 B,
  InMat3 E,
  OutMat C,
  Triangle t);
```

8 *Effects:* Computes $C = E + AB^H + BA^H$.